VAREK — Open Source Data Room

Kenneth Wayne Douglas, MD · MIT License · 2025–2026

> *Synthetic Neural API & Pipeline Scripting Engine — one language for every stage of the AI pipeline.*

File Index

Source Releases

File	Version	Description	Tests	Lines	Size
`varek-v0.1.zip`	v0.1.0	Formal EBNF grammar, lexer, recursive-descent parser, AST, pretty-printer	109	2,546	34 KB
`varek-v0.2.zip`	v0.2.0	Hindley-Milner type inference, Robinson unification, tensor shape tracking, schema validation	163	5,562	60 KB
`varek-v0.3.zip`	v0.3.0	LLVM IR codegen (libLLVM-20 via ctypes), native assembly/object emit, interpreted mode, benchmarks	97	7,792	86 KB
`varek-v0.4.zip`	v0.4.0	Standard library: 7 modules, 261 functions (io / tensor / http / async / pipeline / model / data)	182	11,097	118 KB
`varek-v1.0.zip`	**v1.0.0** ✓	**Stable release. `varek` package manager (20 commands), REPL, formatter, doc gen, governance, 3 RFCs**	**108**	**13,006**	**148 KB**

Cumulative: 659 tests · 13,006 Python lines · 530 KB total

Documentation

File	Description
`VAREK-Spec-Paper-Douglas.pdf`	Formal language specification paper — design rationale, grammar, type system semantics

File	Description
`varek-landing.html`	Project landing page (single-file, GitHub Pages ready)
`VAREK-README.md`	Project README — copy to repo root for GitHub
`VAREK-SPEC.md`	Draft language specification in Markdown (grammar, type rules, built-ins)
`varek-parser.py`	Standalone zero-dependency parser prototype — self-contained reference implementation

---

Version Summary

v0.1 — Foundation

- **EBNF grammar** (`grammar/VAREK.ebnf`) — full formal grammar
- **Lexer** — all tokens with source spans, error recovery
- **Recursive-descent parser** — panic-mode synchronisation, error accumulation
- **AST** — complete node hierarchy + Visitor base class
- **Pretty-printer** — round-trip formatting
- 109 tests, all passing

v0.2 — Type System

- **Hindley-Milner inference** (Algorithm W, let-polymorphism)
- **Robinson's unification** with occurs check
- **Tensor shape tracking** — symbolic dimensions, rank mismatch errors
- **Optional/nullable types** with implicit coercion
- **Schema structural subtyping**
- **Result<T>** and `?` propagation
- Runtime `SchemaValidator`
- 163 tests, all passing

v0.3 — LLVM Backend

- `varek/llvm_api.py` — raw ctypes bindings to libLLVM-20.so (no llvmlite)
- `varek/codegen.py` — AST → verified LLVM IR (SSA, phi nodes, alloca/load/store)

- `varek/compiler.py` — emit-ir / emit-asm / emit-obj / compile pipeline
- **Singleton TargetMachine** (x86 init is not re-entrant)
- `varek/runtime.py` — full tree-walking interpreter retained
- Parse + type check: ~300µs · IR generation: ~1–4ms
- 97 tests, all passing

v0.4 — Standard Library

7 modules, 261 exported functions, lazy-loaded:

Module	Import	Functions	Highlights
`var::io`	`import var::io`	41	read/write/gzip, path ops, env, temp
`var::tensor`	`import var::tensor`	111	NumPy-backed, matmul, SVD, activations, distances
`var::http`	`import var::http`	19	GET/POST/PUT/DELETE, URL ops, JSON
`var::async`	`import var::async as aio`	32	channels, futures, parallel_map, mutex, atomic
`var::pipeline`	`import var::pipeline as pl`	15	run/batch/parallel/stream, chain, retry, cache
`var::model`	`import var::model`	14	inference, embeddings, tokenize, save/load
`var::data`	`import var::data`	29	CSV/JSONL/NPY, splits, augment, metrics

182 tests, all passing

v1.0 — Stable Release

`varek` package manager — 20 commands:

```
syn new <n>        Create a new project
syn run            Run the project (interpreted)
syn build          Compile to LLVM IR / native
syn check          Type-check without running
syn test           Run tests
syn repl           Interactive REPL
syn fmt            Format .syn files
syn doc            Generate documentation
syn install        Install dependencies
syn add <pkg>      Add a dependency
syn publish        Publish to registry
syn search <q>     Search the registry
```

Package format:

- `syn.toml` manifest (semver, deps, build config, scripts)
- `syn.lock` lockfile (exact versions + SHA-256 checksums)
- `.synpkg` archives (gzip-tar)
- Local + remote registry support

Tooling:

- REPL: `:type`, `:env`, `:bench`, `:ir`, `:load`, coloured output
- Formatter: deterministic, operator spacing, blank lines between declarations
- Doc gen: Markdown + HTML + `api.json` from `---` doc comments

Governance:

- `docs/GOVERNANCE.md` — RFC lifecycle, contributor ladder, stability guarantees
- `docs/RFC_TEMPLATE.md` — standard template for language proposals
- `rfcs/0001-pipeline-type-verification.md` — Implemented
- `rfcs/0002-tensor-shape-inference.md` — Implemented
- `rfcs/0003-package-format.md` — Implemented

108 tests, all passing

Quick Start

```bash
# Unzip the latest release
unzip varek-v1.0.zip
cd varek-v1.0

# Create a project
python varek_cli.py new my-project
cd my-project

# Run it
python ../varek_cli.py run

# Start the REPL
python ../varek_cli.py repl

# Type-check
python ../varek_cli.py check
```

Requirements: Python 3.10+ · NumPy · libLLVM-20 (optional, for native emit)

Architecture

```
VAREK Compiler Pipeline
──────────────────────────

source.syn
  │
  ├─ [Lexer]        tokens (varek/lexer.py)
  ├─ [Parser]       AST    (varek/parser.py)
  ├─ [TypeChecker]  types  (varek/checker.py, infer.py, unify.py)
  │
  ├─ [Interpreter]  direct execution  (varek/runtime.py)
  │
  └─ [CodeGen]      LLVM IR  (varek/codegen.py)
        │
        ├─ LLVMVerifyModule  ← IR validation
        ├─ LLVMRunPasses     ← optimization (O0-O3)
        ├─ EmitAssembly      ← .s file
        └─ EmitObject        ← .o file → link → executable
```

```
Standard Library
────────────────

varek/stdlib/
  __init__.py   ← lazy module registry + import resolver
  io.py         ← var::io  (41 fns)
  tensor.py     ← var::tensor (111 fns, NumPy)
  http.py       ← var::http (19 fns)
  async_.py     ← var::async (32 fns)
  pipeline.py   ← var::pipeline (15 fns)
  model.py      ← var::model + var::data (43 fns)
```

License


```
MIT License

Copyright (c) 2025-2026 Kenneth Wayne Douglas, MD

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
```